Runpod Serverless Endpoint Case Study

FLUX.1-dev text-to-image endpoint — build, deployment, and results

Saul Montiel · 18 August 2026 · Repository: github.com/samonti86/runpod-flux-serverless · Endpoint ID: `zj04tmtdmunovm`

Summary

A working Runpod Serverless endpoint that accepts a text prompt and returns a generated 1024×1024 PNG, running **black-forest-labs/FLUX.1-dev** on an H100 80 GB. Generation takes **7.4 seconds**; the endpoint scales to zero between requests.

ITEM	DETAIL
Model	black-forest-labs/FLUX.1-dev (12B params, bf16)
Endpoint	`zj04tmtdmunovm` — Queue type, US-CA-2
Image	Built by Runpod from GitHub; base `pytorch/pytorch:2.5.1-cuda12.4-cudnn9-runtime`
GPU	H100 80 GB SXM · 33.8 GB VRAM in use
Storage	50 GB network volume caching the weights across workers
Code	`handler.py`, `Dockerfile`, `test_endpoint.py`, `test_input.json`

Result

Prompt: "a red fox sitting in tall grass at golden hour, shallow depth of field, photorealistic" · 28 steps · guidance 3.5 · seed 42 · 1024×1024

Input

```
POST https://api.runpod.ai/v2/zj04tmtdmunovm/runsync
{
  "input": {
    "prompt": "a red fox sitting in tall grass at golden hour, shallow depth of field, photorealistic",
    "num_inference_steps": 28,
    "guidance_scale": 3.5,
    "width": 1024,
    "height": 1024,
    "seed": 42
  }
}
```

Output

```
{
  "image_base64": "iVBORw0KGgoAAAANSUhEUg...",   (1488 KB, decoded to output/flux_42.png)
  "seed": 42,
  "parameters": { "model": "black-forest-labs/FLUX.1-dev", "num_inference_steps": 28,
                  "guidance_scale": 3.5, "width": 1024, "height": 1024 },
  "generation_time_seconds": 7.44
}
```

The seed is echoed back deliberately, so any image the endpoint produces can be reproduced exactly.

The design decision that mattered most

The case study asks for *"a Docker image that includes your serverless handler and the model."* I built it that way first. **It cannot be built on Runpod**, and rather than assume that, I established it from the build logs.

FLUX.1-dev is a **gated** model, so downloading it requires an authenticated Hugging Face token — which means the build needs a secret. Two attempts:

ATTEMPT	RESULT
`RUN --mount=type=secret,id=hf_token`	Failed: `ERROR: secret hf_token: not found`
`ARG HF_TOKEN`, with the token set as an endpoint environment variable backed by a Runpod secret	Failed: the step expanded to `RUN HUGGING_FACE_HUB_TOKEN="" ...` — the variable was empty, so endpoint environment variables are not injected into the build

The decisive evidence is the build command Runpod prints in its own logs:

```
/usr/bin/docker buildx build -t registry.runpod.net/samonti86-runpod-flux-serverless-...
  --file .../Dockerfile --network=default --progress=plain
  --output type=oci,dest=... --ulimit cpu=1800 --ulimit nofile=1024:1024
  --cache-to type=local,dest=...
```

> **No `--build-arg`. No `--secret`.** Neither mechanism is passed, so a gated model cannot be authenticated for during a Runpod-side build. That is a platform constraint, not a configuration mistake.

What I did instead, and why

The alternative is to build locally and push. I started that: the model downloaded successfully, but I abandoned the build partway through committing the layer, because a ~41 GB image plus the scratch space needed to compress it

for upload exceeded the free disk on my machine. At a measured 66 Mbps upstream, pushing an image that size is also roughly 80 minutes per iteration. Both are solvable with different hardware; neither is a good use of a three-day window.

So the image carries only the handler and its dependencies — 6.5 GB uncompressed, 3.44 GB as pushed — and the model is fetched once at worker cold start using the `HUGGING_FACE_HUB_TOKEN` secret, cached on a **network volume** so later workers start from cache. That is Runpod's own mechanism for large and gated models.

The trade-off, stated plainly: the first cold start against an empty volume pays a 34 GB download. Every one after that reads from the volume, including on later deployments. Baking the weights in would make that first start as fast as the rest — at the cost of an image Runpod's own builder cannot produce for a gated model.

The slim image *was* built locally and pushed successfully, and is available at `docker.io/samontie86/flux-serverless:v1` as an alternative to the GitHub-built image. The deployed endpoint uses the image Runpod built from the repository.

Measured results

Cold start — three numbers, because they measure different things

BOOT	WORKER	TIME	WHAT IT MEASURES
1st	`t5c3xw34j0crqm`	**177.3 s**	Cold — downloading 34 GB (23 files in 2m06s)
2nd	`t5c3xw34j0crqm`	8.9 s	Same worker restarting, OS page cache still warm
3rd	`f2jgqs4h4xsqvp`	**41.1 s**	**A different worker**, reading from the network volume

> **41.1 s is the honest steady-state figure.** The 8.9 s is flattering but not representative — it is the same machine restarting with the weights still in RAM. The third row is the one that validates the design: a worker that never downloaded the model booted in 41 s because the volume already held it.

End-to-end, measured from the client

```
status: COMPLETED      round-trip: 29.5s
delayTime:      20,728 ms   <- queue wait + worker spin-up
executionTime:   8,198 ms   <- the handler
generation:       7.44 s    <- denoising alone (28 steps @ 4.09 it/s)
```

Seventy percent of that round trip was *acquiring a worker*, not generating. Reporting "30 seconds per image" would be wrong and would point optimisation at the wrong layer — which is why `test_endpoint.py` prints Runpod's `delayTime` and `executionTime` separately rather than just the wall clock.

A second run against the same endpoint made the point sharper. Execution held steady while worker acquisition varied by a factor of eight:

RUN	DELAYTIME	EXECUTIONTIME	GENERATION	ROUND TRIP
1	20,728 ms	8,198 ms	7.44 s	29.5 s
2	**171,202 ms**	8,216 ms	7.40 s	182.3 s

Same code, same endpoint, same prompt length — and a round trip six times longer. None of it was inference: `executionTime` moved by 18 milliseconds. The variance was entirely in acquiring a GPU, and the console was

reporting low supply for the 48 GB class at the time. A user reporting “the endpoint got slower” would be describing capacity, not the model — and only the split metric shows that.

Cost

Billed per millisecond at $0.00133/s ($4.79/hr) with scale-to-zero, so the endpoint costs nothing between requests. A cold start plus one generation is roughly $0.07.

Handler design

The model loads at module scope, not inside `handler()`

A serverless worker is a long-lived process that serves many jobs. Module scope runs once per worker boot; `handler()` runs once per request. Loading a 34 GB model inside the handler would work, but every request would pay the load cost instead of only the first — the serverless equivalent of opening a database connection per query instead of using a pool.

`HF_HOME` is set before `huggingface_hub` is imported

It is read into module constants at import time, so setting it afterwards silently does nothing. That ordering bug produces a correct-looking configuration and a full re-download on every boot. The handler points it at `/runpod-volume` when a network volume is mounted and falls back to local storage when one is not.

Input is clamped, not trusted

The endpoint is reachable once deployed, so every numeric field is bounded: steps 1–50, dimensions ≤1536 and rounded to a multiple of 16 (FLUX's VAE requires it), guidance 0–20. An unclamped `num_inference_steps` is a simple way for a caller to hold a GPU — and a bill — open indefinitely.

Errors return, they do not raise

A bad request returns `{"error": "..."}` so the caller gets an actionable message and the worker stays warm for the next job. CUDA out-of-memory is caught separately because it is the most likely production failure and the fix is caller-side (smaller dimensions) rather than a code change.

Two things I got wrong, and how they surfaced

ISSUE	DIAGNOSIS
`TypeError: FluxPipeline.__call__() got an unexpected keyword argument 'negative_prompt'`	I exposed a `negative_prompt` input the model cannot accept. FLUX.1-dev is *guidance-distilled* — the classifier-free guidance pass a negative prompt steers was trained out — so `FluxPipeline` raises even when the value is `None`. Removed from both the code and the documented API.
`/runsync` returned `IN_QUEUE` after 90 s and looked like a failure	`/runsync` is only mostly synchronous: it blocks for about 90 seconds, then returns the job handle rather than the result. The test client now falls back to polling `/status/<id>`, which is what a cold start requires.

Reproducing this

```
git clone https://github.com/samonti86/runpod-flux-serverless
export RUNPOD_API_KEY=...
export RUNPOD_ENDPOINT_ID=zj04tmtdmunovm
python test_endpoint.py "a red fox in tall grass at golden hour"
```

Writes the PNG to `output/`, prints the seed, and reports the delay/execution split. The endpoint can equally be driven from the console's **Requests** tab or from Postman by POSTing `test_input.json`.

Deployment settings that matter

SETTING	VALUE	WHY
Endpoint type	Queue	Matches the `handler()` contract; Load balancer is for workers running their own HTTP server
GPU	48 GB minimum	Measured at 33.8 GB in use. The quickstart suggests 16 GB, which is right for its example handler — that one loads no model
Active workers	0	Scale to zero; pay only while generating
Idle timeout	120 s	Long enough that consecutive requests reuse a warm worker
Env var	`HUGGING_FACE_HUB_TOKEN` → Runpod secret	Gated model; referencing a secret keeps the token out of plain configuration
Network volume	50 GB attached	Caches weights across workers — the difference between a 177 s and a 41 s cold start

What I would do differently with more time

- **Return an object-storage URL instead of base64.** Inline base64 keeps the endpoint self-contained and easy to test, but inflates the payload ~33% and pushes every byte through the API gateway. At volume, writing to S3 and returning a URL is the better shape.
- **Warm a fresh volume deliberately.** Only the first request against an empty volume pays the download — later deployments reuse it — but that one request is a real user waiting three minutes. A warm-up job run once at provisioning time would move that cost off them.
- **Build where secrets are supported** if baking the weights in were a hard requirement — local BuildKit or a CI runner can mount a secret, and the resulting image would be pushed to a registry for Runpod to pull. That restores a fast first cold start at the cost of a ~41 GB image and a slow build-and-push cycle. The registry is not the constraint; the builder is.

A note on how this was built

I wrote this with AI coding assistance, which is how I build tooling — I specify what needs to exist, drive the assistant, then read, test and debug the result. I am stating that plainly because it is how the work was actually done. The design decisions documented here are ones I made and can defend: where the model loads, why `HF_HOME` precedes the import, why inputs are clamped, why the GPU is sized to the model rather than to the tutorial, and why the weights are fetched at cold start rather than baked in. The two failures above were diagnosed by reading build logs and tracebacks, not by guessing.